

```

Doub dual_obj=dotprod(b,y);
Doub gamma_norm=gamma/(1.0+abs(primal_obj));
  cout << setw(3) << iter << setw(12) << primal_obj << setw(12) <<
    normrp << setw(12) << dual_obj << setw(12) << normrd << setw(12)
    << gamma << setw(12) << gamma_norm<<endl;
if (normrp < EPS && normrd < EPS && gamma_norm < EPS)
  return status=0;           Optimal solution found.
if (normrp > 1000*normrp_old && normrp > EPS)
  return status=1;           Primal infeasible.
if (normrd > 1000*normrd_old && normrd > EPS)
  return status=2;           Dual infeasible.
for (j=0;j<n;j++)             Compute step directions. First form matrix
  d[j]=x[j]/z[j];               $A \cdot X \cdot Z^{-1} \cdot A^T$ .
adat.updateD(d);
solver.factorize();            Factorize matrix.
for (j=0;j<n;j++)             Form right-hand side.
  tempn[j]=x[j]-mu/z[j]-d[j]*rd[j];
tempm=a.ax(tempn);
for (i=0;i<m;i++)
  rhs[i]=-rp[i]+tempm[i];
solver.solve(dy,rhs);         Solve for  $dy$ .
tempn=at.ax(dy);              Solve for  $dz$ .
for (j=0;j<n;j++)
  dz[j]=-tempn[j]-rd[j];
for (j=0;j<n;j++)             Solve for  $dx$ .
  dx[j]=-d[j]*dz[j]+mu/z[j]-x[j];
Doub alpha_p=1.0;             Find step length.
for (j=0;j<n;j++)
  if (x[j]+alpha_p*dx[j] < 0.0)
    alpha_p=-x[j]/dx[j];
Doub alpha_d=1.0;
for (j=0;j<n;j++)
  if (z[j]+alpha_d*dz[j] < 0.0)
    alpha_d=-z[j]/dz[j];
alpha_p = MIN(alpha_p*SIGMA,1.0);
alpha_d = MIN(alpha_d*SIGMA,1.0);
for (j=0;j<n;j++) {           Step to new point.
  x[j]+=alpha_p*dx[j];
  z[j]+=alpha_d*dz[j];
}
for (i=0;i<m;i++)
  y[i]+=alpha_d*dy[i];
normrp_old=normrp;            Update norms.
normrd_old=normrd;
}
return status=3;               Maximum iterations exceeded.
}

```

10.11.7 Practical Interior-Point Codes

There are a number of important features that would be needed to turn the above simple implementation into a state-of-the-art code.

- Initial point. Choosing a good starting point cuts down the number of iterations required. A good algorithm is described in [7].
- Preprocessing. As for the simplex method, preprocessing can often reduce the size of the problem.
- Scaling. A badly scaled problem can lead to numerical difficulties.
- Handling bounded variables. Suppose that instead of the requirement $\mathbf{x} \geq 0$ the variables are *bounded*:

$$\mathbf{l} \leq \mathbf{x} \leq \mathbf{u} \quad (10.11.27)$$

Here, for simplicity, we have written the vectors of lower and upper bounds, $\mathbf{l}$ and $\mathbf{u}$, as being of length n . In practice, only some of the variables $\mathbf{x}$ may have bounds. One way to deal with bounds is to add them to the system $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$ with slack variables in the usual way. However, this increases the dimension of the matrix $\mathbf{A}$. There is a simpler way to proceed. First, lower bounds of the form $l_j \leq x_j$ can be handled by a simple shift: $x'_j = x_j - l_j \geq 0$. Making this replacement everywhere allows the problem to be solved as before, and then you simply undo the shift to get the solution in terms of the original x_j . So without loss of generality we can assume all the bounds are of the form

$$0 \leq \mathbf{x} \leq \mathbf{u} \quad (10.11.28)$$

If we introduce slack variables $\mathbf{s}$ and dual slack variables $\mathbf{w}$, the optimality conditions are

$$\begin{aligned} \mathbf{A} \cdot \mathbf{x} &= \mathbf{b} \\ \mathbf{x} + \mathbf{s} &= \mathbf{u} \\ \mathbf{A}^T \cdot \mathbf{y} + \mathbf{z} - \mathbf{w} &= \mathbf{c} \\ \mathbf{X} \cdot \mathbf{Z} \cdot \mathbf{e} &= 0 \\ \mathbf{S} \cdot \mathbf{W} \cdot \mathbf{e} &= 0 \end{aligned} \quad (10.11.29)$$

with $\mathbf{x}$, $\mathbf{s}$, $\mathbf{z}$, and $\mathbf{w}$ all nonnegative. It is simple to change the right-hand sides of the last two equations in (10.11.29) to $\tau \mathbf{e}$ and apply Newton's method as for equation (10.11.11). You find that the equations to be solved are very similar in form to equations (10.11.23) – (10.11.25).

- **Predictor-corrector.** Most of the time spent in an iteration goes into the factorization of the matrix in the normal equations. Given the factorization, the solve step is relatively cheap. Mehotra's predictor-corrector method [7] takes advantage of this by using an extra solve step per iteration to improve the overall efficiency of the algorithm.

Recall that in going from equation (10.11.19) to equation (10.11.20) we dropped the term $\Delta \mathbf{X} \cdot \Delta \mathbf{Z} \cdot \mathbf{e}$. The idea in Mehotra's method is to first take a predictor step that solves equation (10.11.20), but with the τ term omitted. The values of $(\Delta \mathbf{x}, \Delta \mathbf{y}, \Delta \mathbf{z})$ obtained are used to estimate $\Delta \mathbf{X} \cdot \Delta \mathbf{Z} \cdot \mathbf{e}$. Then the corrector step solves for an additional set $(\Delta \mathbf{x}, \Delta \mathbf{y}, \Delta \mathbf{z})$ from equation (10.11.20) with the right-hand side replaced by

$$\begin{bmatrix} 0 \\ 0 \\ \tau \mathbf{e} - \Delta \mathbf{X} \cdot \Delta \mathbf{Z} \cdot \mathbf{e} \end{bmatrix} \quad (10.11.30)$$

The value of τ in equation (10.11.30) is set differently from equation (10.11.12). First $\hat{\mu}$ is computed using the predictor step:

$$\hat{\mu} = (\mathbf{x} + \alpha_p \Delta \mathbf{x}) \cdot (\mathbf{z} + \alpha_d \Delta \mathbf{z}) / n \quad (10.11.31)$$

Here α_p and α_d are the largest values that maintain nonnegativity, but no larger than unity. (No safety factor is used.) Then τ is set as

$$\tau = \left(\frac{\hat{\mu}}{\mu} \right)^2 \hat{\mu} \quad (10.11.32)$$

where μ is computed using the starting values of $\mathbf{x}$ and $\mathbf{z}$ as in equation (10.11.12). This heuristic choice makes τ small when the predictor step gives a large decrease in complementarity, and large otherwise.

The total step is the sum of the predictor and corrector steps. The cutdown factor from α_p or α_d equal to unity is calculated from a heuristic procedure described in [7] or [8].

Gondzio [9,10] has developed an extension to the predictor-corrector algorithm that incorporates higher-order corrections when they can improve the efficiency.

- Better sparse matrix algebra. While AMD is a good general-purpose choice for an ordering algorithm, LDL is a good but basic sparse Cholesky routine, chosen mainly for its simplicity and availability. More powerful algorithms are known and are starting to become publicly available.

One of the problems with the normal equations is that the matrix can be quite dense, even when $\mathbf{A}$ itself is rather sparse. This has motivated algorithms that solve the augmented equations (10.11.22) directly. On some problems, this leads to significant savings. A good implementation will provide both alternatives.

Solving the equations can become numerically delicate, especially as the optimal point is approached. Good implementations will use some form of iterative refinement to preserve accuracy.

- Crossover to the simplex method. Often the convergence of the interior-point algorithm slows down near the optimal point. By switching to a simplex method with a basis that is presumably close to optimal, one can get rapid convergence to the answer. This feature has the additional benefit that the optimal point is given in terms of basis vectors, whereas the interior-point solution never actually attains zeros for any $\mathbf{x}$'s. Some kinds of post-analysis need the actual basis.

Interestingly, using a previously-found solution as the initial point for a “nearby” problem seldom helps much for interior-point methods. The reason is that interior-point methods don’t make good progress from a point near the boundary. The simplex method, by contrast, generally converges much more rapidly with a “warm start.” A good strategy for solving a sequence of closely related problems is therefore interior-point with crossover to an optimal basis for the first one, then simplex with a warm start for the remainder.

There are several codes that are free for noncommercial use and that give full implementations of interior-point methods. We particularly like PCx (in C with Fortran sparse algebra routines) [11] and HOPDM (in Fortran) [10]. For a discussion of more options, including commercial codes, see [12].

So which should you use: a simplex or an interior-point code? If you have only our codes, use the interior-point one. If you have a production implementation of either algorithm, it will probably suffice. If you are solving many large problems, however, you should have both so you can use the optimum algorithm in each case. If you are solving a large problem for the first time, there is a lot to be said for using an interior-point code. There are fewer choices to make to get almost optimal performance. By contrast, finding which particular choices of components in a simplex method give optimal performance can involve a lot of experimentation. Your first try will usually not be as good as a default interior-point code. And often the

interior-point code will beat all simplex variants.

CITED REFERENCES AND FURTHER READING:

- Khachian, L. 1979, *Doklady Akademii Nauk SSSR*, vol. 244, pp. 191–194. English translation: “A Polynomial Time Algorithm in Linear Programming,” *Soviet Mathematics Doklady*, vol. 20, pp. 191–194.[1]
- Karmarkar, N. 1984, “A New Polynomial-time Algorithm for Linear Programming,” *Combinatorica* vol. 4, pp. 373–395.[2]
- Maros, I. 2003, *Computational Techniques of the Simplex Method* (Boston: Kluwer).[3]
- Nazareth, J.L. 2004, *An Optimization Primer* (New York: Springer).[4]
- LDL, <http://www.cise.ufl.edu/research/sparse>. [5]
- AMD, <http://www.cise.ufl.edu/research/sparse>. See also Amestoy, P.R., Enseeihlrit, Davis, T.A., and Duff, I.S. 2004, “AMD, an Approximate Minimum Degree Ordering Algorithm,” *ACM Transactions on Mathematical Software* vol. 30, pp. 381–388.[6]
- Mehrotra, S. 1992, “On the Implementation of a Primal-dual Interior Point Method,” *SIAM Journal on Optimization* vol. 2, pp. 575–601.[7]
- Wright, S.J. 1997, *Primal-Dual Interior-Point Methods* (Philadelphia: S.I.A.M.).[8]
- Gondzio, J. 1996, “Multiple Centrality Corrections in a Primal-dual Method for Linear Programming,” *Computational Optimization and Applications* vol. 6, pp. 137–156.[9]
- HOPDM, <http://www.maths.ed.ac.uk/~gondzio/software/hopdm.html>. [10]
- PCX, <http://www-fp.mcs.anl.gov/otc/Tools/PCx>. [11]
- Linear Programming Frequently Asked Questions, <http://www-unix.mcs.anl.gov/otc/Guide/faq/linear-programming-faq.html>. [12]
- Vanderbei, R.J. 2001, *Linear Programming: Foundations and Extensions*, 2nd ed. (Boston: Kluwer). [Undergraduate/graduate text]
- Numerical Recipes Software 2007, “Interface to AMD and LDL Packages,” *Numerical Recipes Webnote No. 13*, at <http://www.nr.com/webnotes?13> [13]

10.12 Simulated Annealing Methods

The *method of simulated annealing* [1,2] is a technique that has attracted significant attention as suitable for optimization problems of large scale, especially ones where a desired global extremum is hidden among many poorer, local extrema. For practical purposes, simulated annealing has effectively “solved” the famous *traveling salesman problem* of finding the shortest cyclical itinerary for a traveling salesman who must visit each of N cities in turn. (Other practical methods have also been found.) The method has also been used successfully for designing complex integrated circuits: The arrangement of several hundred thousand circuit elements on a tiny silicon substrate is optimized so as to minimize interference among their connecting wires [3,4]. Surprisingly, the implementation of the algorithm is relatively simple.

Notice that the two applications cited are both examples of *combinatorial minimization*. There is an objective function to be minimized, as usual, but the space over which that function is defined is not simply the N -dimensional space of N continuously variable parameters. Rather, it is a discrete, but very large, configuration space, like the set of possible orders of cities, or the set of possible allocations of silicon

“real estate” blocks to circuit elements. The number of elements in the configuration space is factorially large, so that they cannot be explored exhaustively. Furthermore, since the set is discrete, we are deprived of any notion of “continuing downhill in a favorable direction.” The concept of “direction” may not have any meaning in the configuration space.

Below, we will also discuss how to use simulated annealing methods for spaces with continuous control parameters, like those of §10.5 – §10.9. This application is actually more complicated than the combinatorial one, since the familiar problem of “long, narrow valleys” again asserts itself. Simulated annealing, as we will see, tries “random” steps; but in a long, narrow valley, almost all random steps are uphill! Some additional finesse is therefore required.

At the heart of the method of simulated annealing is an analogy with thermodynamics, specifically with the way that liquids freeze and crystallize or metals cool and anneal. At high temperatures, the molecules of a liquid move freely with respect to one another. If the liquid is cooled slowly, thermal mobility is lost. The atoms are often able to line themselves up and form a pure crystal that is completely ordered over a distance up to billions of times the size of an individual atom in all directions. This crystal is the state of minimum energy for this system. The amazing fact is that, for slowly cooled systems, nature is able to find this minimum energy state. In fact, if a liquid metal is cooled quickly or “quenched,” it does not reach this state but rather ends up in a polycrystalline or amorphous state having somewhat higher energy.

So the essence of the process is *slow* cooling, allowing ample time for the redistribution of the atoms as they lose mobility. This is the technical definition of *annealing*, and it is essential for ensuring that a low energy state will be achieved.

Although the analogy is not perfect, there is a sense in which all of the minimization algorithms thus far in this chapter correspond to rapid cooling or quenching. In all cases, we have gone greedily for the quick, nearby solution: From the starting point, go immediately downhill as far as you can go. This, as often remarked above, leads to a local, but not necessarily a global, minimum. Nature’s own minimization algorithm is based on quite a different procedure. The so-called Boltzmann probability distribution,

$$\text{Prob}(E) \sim \exp(-E/kT) \quad (10.12.1)$$

expresses the idea that a system in thermal equilibrium at temperature T has its energy probabilistically distributed among all different energy states E . Even at low temperature, there is a chance, albeit a very small one, of a system being in a high energy state. Therefore, there is a corresponding chance for the system to get out of a local energy minimum in favor of finding a better, more global one. The quantity k (Boltzmann’s constant) is a constant of nature that relates temperature to energy. In other words, the system sometimes goes *uphill* as well as downhill; but the lower the temperature, the less likely is any significant uphill excursion.

In 1953, Metropolis and coworkers [5] first incorporated these kinds of principles into numerical calculations. Offered a succession of options, a simulated thermodynamic system was assumed to change its configuration from energy E_1 to energy E_2 with probability $p = \exp[-(E_2 - E_1)/kT]$. Notice that if $E_2 < E_1$, this probability is greater than unity; in such cases the change is arbitrarily assigned a probability $p = 1$, i.e., the system *always* took such an option. This general scheme, of always taking a downhill step while *sometimes* taking an uphill step, has come to be known as the Metropolis algorithm.

To make use of the Metropolis algorithm for other than thermodynamic systems, one must provide the following elements:

1. A description of possible system configurations.
2. A generator of random changes in the configuration; these changes are the “options” presented to the system.
3. An objective function E (analog of energy) whose minimization is the goal of the procedure.
4. A control parameter T (analog of temperature) and an *annealing schedule* that tells how it is lowered from high to low values, e.g., after how many random changes in configuration is each downward step in T taken, and how large is that step. The meaning of “high” and “low” in this context, and the assignment of a schedule, may require physical insight and/or trial-and-error experiments.

We will return to these ideas in §15.8, with a more rigorous discussion of *Markov chain Monte Carlo* and the *Metropolis-Hastings algorithm*.

10.12.1 Combinatorial Minimization: The Traveling Salesman

A concrete illustration is provided by the traveling salesman problem. The proverbial seller visits N cities with given positions (x_i, y_i) , returning finally to his or her city of origin. Each city is to be visited only once, and the route is to be made as short as possible. This problem belongs to a class known as *NP-complete* problems, whose computation time for an *exact* solution increases with N as $\exp(\text{const.} \times N)$, becoming rapidly prohibitive in cost as N increases. The traveling salesman problem also belongs to a class of minimization problems for which the objective function E has many local minima. In practical cases, it is often enough to be able to choose from these a minimum that, even if not absolute, cannot be significantly improved upon. The annealing method manages to achieve this, while limiting its calculations to scale as a small power of N .

As a problem in simulated annealing, the traveling salesman problem is handled as follows:

1. *Configuration.* The cities are numbered $i = 0 \dots N - 1$ and each has coordinates (x_i, y_i) . A configuration is a permutation of the number $0 \dots N - 1$, interpreted as the order in which the cities are visited.
2. *Rearrangements.* An efficient set of moves has been suggested by Lin [6]. The moves consist of two types: (i) A section of path is removed and then replaced with the same cities running in the opposite order; or (ii) a section of path is removed and then replaced in between two cities on another, randomly chosen, part of the path.
3. *Objective function.* In the simplest form of the problem, E is taken just as the total length of the journey,

$$E = L \equiv \sum_{i=0}^{N-1} \sqrt{(x_i - x_{i+1})^2 + (y_i - y_{i+1})^2} \quad (10.12.2)$$

with the convention that point N is identified with point 0. To illustrate the flexibility of the method, however, we can add the following additional wrinkle: Suppose that the salesman has an irrational fear of flying over the Missis-